

4. Bölüm

C++ Programlama Diline Giriş

4.1 En Basit C++ Programı

C programlama dili, her programcının ihtiyacını görecektir genel amaçlı, emreden paradigmalı, yapısal programlama dilidir. Kısa ve öz talimatlar birbiri ardına verilerek yapılan programlamaya emreden paradigma (**imperative programming**) adı verilir. Yapısal programlamanın çerçevesi 2.7 başlığı altında anlatılmıştır. Yapısal programlamada, programın başladığı yeri belirten bir **ana fonksiyon (main function)** tanımlanır. C dilinde bu fonksiyon **main()** fonksiyonudur. C++ dili, C programlama diline, nesne yönelimli programlama mantığını kodlayabilmesi için **eklemler yapılarak geliştirilmiştir**. C++ dilinde de ana fonksiyon, yine **main()** fonksiyonudur. Yani içinde **main()** fonksiyonu yazılmamış bir C++ programı çalışmaz. C++ dilinde de talimatlar noktalı virgül karakteri ile bittiğinden **gelişigüzel (free format)** yazılır. Aşağıda en basit C++ programı verilmiştir.

```
1 int main()
2 {
3     return 0;
4 }
```

Kodu şu şekilde açıklayabiliriz;

- ♦ Birinci satırda, yapısal programlamanın **ana fonksiyonu (main function)** tanımlanmıştır. Başındaki **int**, fonksiyonun bir tam sayı (**integer**) geri döndüreceğini belirtir. Program çalışmaya ana fonksiyondan başlar ve bittiğinde işletim sistemine geri döner.
- ♦ C dilinde her fonksiyon ikinci satırdaki gibi süslü parantez karakteri ile başlayan ve dördüncü satırdaki gibi biten bir **kod bloğuna (code block)** sahiptir. Her fonksiyonda olan bu blok, **fonksiyon bloğu (function block)** olarak da adlandırılır.
- ♦ Ana fonksiyon içinde üçüncü satırda yazılan **return 0;** talimatı (**statement**) ana fonksiyondan sıfır geri döndürerek çıktıldığını söyleyen talimattır. Sıfır dışında bir değer işletim sistemine gönderilmesi, programın hata ile sonlandığı olarak kabul edilir. Eğer programdan çıkışta işletim sistemine **11** göndermek istersek bu talimatı **return 11;** olarak yazmalıyız. Ana fonksiyonda varsayılan olarak, hiçbir hata ile karşılaşmadığı için, işletim sistemine **0** gönderilir. Bu nedenle **return 0;** talimatının yazılması zorunlu değildir. Ancak başka bir değer gönderilecek ise yazılır.

4.2 Söz Dizim Kuralları

C++ dili için oluşturulacak kaynak kod programcı tarafından metin dosyasına yazılırken belli **söz dizim kurallarına (syntax)** uyar. Bunlar aşağıda başlıklar halinde verilmiştir.

§4.2.1 Kaynak Kodumuzu Oluşturacak Karakterler

1. İngiliz alfabesindeki büyük harfler:
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
2. İngiliz Alfabesindeki küçük harfler:
a b c d e f g h i j k l m n o p q r s t u v w x y z
3. Rakamlar:
0 1 2 3 4 5 6 7 8 9
4. Özel Karakterler:
< > . , _ () ; \$: % [] # ? ' & { } " ^ ! * / | - ~ +
5. Metinde görünmeyen ancak metni biçimlendiren **beyaz boşluk (white space)** karakterleri;
 - (a) Boşluk (**space**)
 - (b) Yeni satır (**new line**)
 - (c) Satır başı (**carriage return**)
 - (d) Geri (**backspace**)
 - (e) Sekme (**tab**)

§4.2.2 Talimatlar ve Anahtar Kelimeler

Talimat

Yüksek seviyeli bir dilde yazılmış bir **talimat** (**statement**), işlemciye belirtilen bir eylemi gerçekleştirmesini söyleyen cümledir. Yüksek seviyeli bir dildeki tek bir talimat, birkaç emir kodunu (**instruction code**) temsil edebilir. Talimatlar, yapılması gerekeni kısa ve net olarak belirten mantıksal satırlardır (**logical sequence**).

Anahtar kelimeler (**keywords**), programlamada kullanılan ve C++ derleyicisi tarafından özel anlama sahip, önceden tanımlanmış, **ayrılmış kelimeler** (**reserved words**) olup programı oluşturan **talimatlar** (**statement**) bu kelimeler kullanılarak yazılırlar¹. Tablo 4.1’de anahtar kelimeler verilmiştir.

<code>alignas</code> ⁽¹¹⁾	<code>constexpr</code> ⁽¹¹⁾⁽¹⁷⁾	<code>inline</code> ⁽¹¹⁾⁽¹⁷⁾	<code>static</code>
<code>alignof</code> ⁽¹¹⁾	<code>constinit</code> ⁽²⁰⁾	<code>int</code> ⁽¹¹⁾	<code>static_assert</code> ⁽¹¹⁾
<code>and</code>	<code>const_cast</code>	<code>long</code>	<code>static_cast</code>
<code>and_eq</code>	<code>continue</code>	<code>mutable</code> ⁽¹¹⁾	<code>struct</code> ⁽¹¹⁾
<code>asm</code>	<code>co_await</code> ⁽²⁰⁾	<code>namespace</code>	<code>switch</code>
<code>atomic_cancel</code>	<code>co_return</code> ⁽²⁰⁾	<code>new</code>	<code>synchronized</code>
<code>atomic_commit</code>	<code>co_yield</code> ⁽²⁰⁾	<code>noexcept</code> ⁽¹¹⁾	<code>template</code>
<code>atomic_noexcept</code>	<code>decltype</code> ⁽¹¹⁾⁽¹⁴⁾	<code>not</code>	<code>this</code> ⁽²³⁾
<code>auto</code> ⁽¹¹⁾⁽¹⁷⁾⁽²⁰⁾⁽²³⁾	<code>default</code> ⁽¹¹⁾	<code>not_eq</code>	<code>thread_local</code> ⁽¹¹⁾
<code>bitand</code>	<code>delete</code> ⁽¹¹⁾	<code>nullptr</code> ⁽¹¹⁾	<code>throw</code> ⁽¹⁷⁾⁽²⁰⁾
<code>bitor</code>	<code>do</code>	<code>operator</code> ⁽¹¹⁾	<code>true</code>
<code>bool</code>	<code>double</code>	<code>or</code>	<code>try</code>
<code>break</code>	<code>dynamic_cast</code>	<code>or_eq</code>	<code>typedef</code>
<code>case</code>	<code>else</code>	<code>private</code> ⁽²⁰⁾	<code>typeid</code>
<code>catch</code>	<code>enum</code> ⁽¹¹⁾	<code>protected</code>	<code>typename</code> ⁽¹⁷⁾⁽²⁰⁾
<code>char</code>	<code>explicit</code>	<code>public</code>	<code>union</code>
<code>char8_t</code> ⁽²⁰⁾	<code>export</code> ⁽¹¹⁾⁽²⁰⁾	<code>constexpr</code>	<code>unsigned</code>
<code>char16_t</code> ⁽¹¹⁾	<code>extern</code> ⁽¹¹⁾	<code>register</code> ⁽¹⁷⁾	<code>using</code> ⁽¹¹⁾⁽²⁰⁾
<code>char32_t</code> ⁽¹¹⁾	<code>false</code>	<code>reinterpret_cast</code>	<code>virtual</code>
<code>class</code> ⁽¹¹⁾	<code>float</code>	<code>requires</code> ⁽²⁰⁾	<code>void</code>
<code>compl</code>	<code>for</code> ⁽¹¹⁾	<code>return</code>	<code>volatile</code>
<code>concept</code> ⁽²⁰⁾	<code>friend</code>	<code>short</code>	<code>wchar_t</code>
<code>const</code>	<code>goto</code>	<code>signed</code>	<code>while</code>
<code>constexpr</code> ⁽²⁰⁾⁽²³⁾	<code>if</code> ⁽¹⁷⁾⁽²³⁾	<code>sizeof</code> ⁽¹¹⁾	<code>xor</code>

Tablo 4.1: Anahtar Kelimeler

Talimatlar ve Anahtar Kelimeler

Anahtar kelimeler, küçük-büyük harf ayrımı yapıldığından **her zaman küçük harflerle yazılırlar**. Her **talimat** (**statement**), anahtar kelimeler içerecek şekilde yazılır ve **noktalı virgül karakteri ile biter**.

Talimatlar BASIC ve FORTRAN gibi dillerde genellikle satır sonunda biter. FORTRAN dilinde yedinci sütundan yazılmaya başlanır. BASIC dilinde ise her satırın başına satır numarası yazılır. Pascal ve C dillerinde böyle bir kural söz konusu değildir. Talimat herhangi bir yerde başlar ve noktalı virgül karakteri ile biter. Aradaki beyaz boşluklar ne kadar çok olursa olsun tek bir tane varmış gibi kabul edilir. Bu nedenle kod **gelişigüzel** (**free format**) yazılır.

Aşağıda gelişigüzel yazılan programların hepsi derlenebilir ve derleme sonucu aynı çalıştırılabilir dosyayı üretir. Beyaz boşlukları alınmış en kısa kod aşağıda verilmiştir:

```
int main() {}
```

Kod içerisinde beyaz boşluk karakterleri anahtar kelimeleri bölmediği sürece istenildiği kadar kullanılabilir:

```
int
main ( ) {
    return    0;
```

¹<https://en.cppreference.com/w/cpp/keyword> İlgili uyarlamasında değişti veya eklendi anlamındadır.

```
}
```

Fonksiyon parantezleri içine veya fonksiyon blokları içine istenildiği kadar beyaz boşluk konulabilir. Parantezler ve fonksiyon blokları hizalı olmayabilir:

```
int main (
{
}
```

Kodun güzel görünmesi şart değildir:

```
int main(
)
{
    return
    0;
}
```

Ancak kodun bir başkası tarafından bakımı yapılacağı göz önüne alınarak okunaklı yazılması çok önemlidir.

En doğrusu programı aşağıdaki sitilde yazmaktır;

- ◊ Fonksiyonların gövdesi ve bazı kontrol talimatları kod bloklarından oluşur. Kod blokları için süslü parantez kullanılır.
- ◊ Süslü parantez içindeki kodlar, okunabilirliği artırmak için genellikle **4 boşluk** veya bir **sekme (tab)** içeriden başlar. Kodu bu şekilde yazmaya **girintili (indented)** yazma adı verilir.
- ◊ Ayrıca değişkenler ve anahtar kelimeler arasında boşluk karakteri veya sekme karakteri kullanılmalıdır:

```
int main() {
//Ana fonksiyon bloğuna ait girinti
int sayac=0;
if (sayac==0) { //iç blok
//iç bloğa ait girinti...
int sonuc=1;
return sonuc;
}
if (sayac==1) // Blok başlangıcı böyle de yazılabilir
{ //iç blok
//iç bloğa ait girinti...
int sonuc=1;
return sonuc;
}
}
```

§4.2.3 Açıklamalar

Talimatlar (statement), yapılması gerekeni kısa ve net olarak belirten **mantıksal satırlardır (logical sequence)**. Programcı kodu yazarken **açıklama (comment)** yapma gereği duyabilir. Bunu iki şekilde yapar;

1. Kod metninde bulunulan yerden sonra satırın sonuna kadar olan bir açıklama yapılmak istendiğinde, açıklama öncesinde iki adet bölü (//) karakteri kullanılır.
2. Eğer birden çok satırı içeren bir açıklama yapılacak ise açıklama (/*) ile (*/) karakterleri arasına alınır.

Açıklamalar

İstedikimiz kadar açıklama yapabiliriz. Derleyici açısından bunun önemi yoktur. Çünkü derlemenin ilk aşamasında **bütün bu açıklamalar ve beyaz boşluk karakterleri metinden çıkartılır.**

Aşağıda açıklamalar eklenmiş ana fonksiyon örneği yer almaktadır;

```
/*
Bu program, açıklama (comment) içeren basit bir ana fonksiyonu olan C++ programıdır.
İlhan ÖZKAN, Ocak 2025
*/
int main() // Ana Fonksiyon
{ //fonksiyon bloğu başlagıcı
/*
Ana Fonksiyon, programın başladığı yerdir. Ana fonksiyonu olmayan bir program çalışmaz.
*/
```

```
} //fonksiyon bloğu bitişi
```

4.3 Değişken Tanımlama

Her yapısal programlama dilinde olduğu gibi C dilinde de **veri yapıları** (**data structure**) ve **kontrol yapıları** (**control structure**) birbirinden ayrıdır. Dolayısıyla veriyi işleyecek kontrol yapısı kodlanmadan önce veriyi taşıyan veri yapıları tanımlanmalıdır.

Değişkenler (**variable**) veri yapılarının temel öğeleridir. Bu başlıkta anlatılan tam sayı ve kayan noktalı sayı değişkenleri, belleğin belli bir bölgesini işgal eder ve fiziki olarak bellek bitlerinden oluştuğundan ikili sayı (**binary**) olarak veri tutarlar.

§4.3.1 Tam Sayı Veri Tipleri

C++ dilinde 2.4 başlığında anlatılan tam sayılar (**integer**), **ilkel veri tiplerinden** (**primitive data type**) biridir. Tam sayı veri tipleri için aşağıdaki anahtar kelimeler kullanılır;

Veri Tipi	Boyut (Byte/Bit)	Kullanım Amacı / Özellik	Mimari Notu
<code>char</code>	1 Bayt (8 Bit)	İşaretsiz tam sayı / Karakter	En küçük adreslenebilir birim
<code>short</code>	2 Bayt (16 Bit)	Küçük ölçekli tam sayılar	Bellek tasarrufu sağlar
<code>int</code>	2/4 Bayt (16/32 Bit)	Standart tam sayı	Sistem mimarisine göre değişir
<code>long</code>	4/8 Bayt (32/64 Bit)	Geniş tam sayı	Sistem mimarisine göre değişir
<code>long long</code>	8 Bayt (64 Bit)	Çok geniş tam sayı	Modern 64-bit hesaplamalar
<code>wchar_t</code>	2/4 Bayt (16/32 Bit)	Çok dilli (wide) metinler	Uluslararası karakterler
<code>char16_t</code>	2 Bayt (16 Bit)	UTF-16 Karakterler	Taşınabilir çok dilli destek
<code>char32_t</code>	4 Bayt (32 Bit)	UTF-32 Karakterler	Tam Unicode karakter desteği
<code>bool</code>	1 Bayt (8 Bit)	Mantıksal (true / false)	Arka planda 1 ve 0
<code>std::size_t</code>	Mimari Bağımlı	Nesne boyutu / İndis	sizeof işlecinin sonucudur

Tablo 4.2: Tam Sayı Veri Tipleri

§4.3.2 Kayan Noktalı Sayı Veri Tipleri

C++ dilinde 2.4 başlığında anlatılan kesirli sayılar için kullanılan kayan noktalı sayı (**floating point number**), **ilkel veri tiplerinden** (**primitive data type**) biridir ve bu veri tipleri için aşağıdaki anahtar kelimeler kullanılır;

Veri Tipi	Boyut (Byte/Bit)	Hassasiyet (Precision)	Kullanım Alanı
<code>float</code>	4 Bayt (32 Bit)	Tek Hassasiyet (Single)	Grafik / Düşük hassasiyetli işler
<code>double</code>	8 Bayt (64 Bit)	Çift Hassasiyet (Double)	Bilimsel ve genel hesaplamalar
<code>long double</code>	8/16 Bayt (64/128 Bit)	Çok Hassasiyetli	CLang/GCC ve Spark, PowerPC gibi bazı platformlarda <code>long double</code> veri tipi 16 bayt (128 bit) iken, diğer platformlarda 8 bayt (64 bit) olabilir. Bu nedenle, <code>sizeof(long double)</code> ifadesi platforma bağlı olarak farklı sonuçlar verebilir.

Tablo 4.3: Kayan Noktalı Sayı Veri Tipleri

§4.3.3 Diğer Veri Tipleri

Tam sayılar 4.2 başlığında anlatıldığı üzere standart tiplerin (`int`, `long` vb.) boyutları platformdan platforma değişebilir. Profesyonel projelerde tam kontrol sağlamak için `<stdint>` başlığındaki sabit genişlikli tipler kullanılmalıdır. Bunlar;

- ◊ `int8_t` / `uint8_t`: Her platformda geçerli tam 8 Bit Tam Sayı
- ◊ `int16_t` / `uint16_t`: Her platformda geçerli tam 16 Bit Tam Sayı
- ◊ `int32_t` / `uint32_t`: Her platformda geçerli tam 32 Bit Tam Sayı
- ◊ `int64_t` / `uint64_t`: Her platformda geçerli tam 64 Bit Tam Sayı

Bunların dışında ileride yeri geldikçe göreceğimiz tam sayı gösterici (`std::intptr_t`) ve işaretsiz tam sayı gösterici (`std::uintptr_t`) tipleri de vardır. Bu, düşük seviyeli sistem programlaması için çok önemlidir.

void Anahtar Kelimesi

C++ programlama dilinde bir de **hiçbir değeri olmayan ve bellekte yer kaplamayan void** veri tipi vardır. Bu veri tipini hiçbir değer döndürmeyen fonksiyon tanımlamalarında ve dinamik bellek kulllanımlarında ilerde çokça göreceğiz.

§4.3.4 Değişken Tanımlama

Değişkenlerin veri tipine (**data type**) bağlı olarak benzersiz bir **kimlik (identifier)** verilerek **tanımlanması (definition)** gerekir.

```
/* Bu program değişken tanımlamak için oluşturulmuştur.*/
int main() {
    char yas; /* veri tipi "char" ve kimliği "yas" olan değişken tanımlandı. -128 ila +127
    ↪ arasında değer alabilir. */
    short kat; /* veri tipi "short" ve kimliği "kat" olan değişken tanımlandı. -32768 ila +32768
    ↪ arasında değer alabilir. */
    float kilo; /* veri tipi "float" ve kimliği "kilo" olan değişken tanımlandı. */
}
```

Yukarıda tanımlaması yapılan **yas**, **kat** ve **kilo** değişkenleri ana fonksiyon bloğu içinde tanımlandığından, tanımlanan değişkenler sadece bu blok içinde geçerlidir!

Değişken Kimliklendirme

Aynı blok içinde bir değişkene verilen kimlik, bir başka değişkene verilemez!

Bildirim (declaration) derleyiciye böyle bir değişken, fonksiyon veya nesne var demektir. Bunu ilerde daha detaylı göreceğiz. Şimdilik tanımlama gibi kabul edebilirsiniz. **Tanımlama, (definition)** derlemenin bağlama (**link**) ile birlikte sorunsuz olabilecek şekilde eksiksiz yapılan bir işlemdir.

§4.3.5 Değişken Kimliklendirme Kuralları

Kimliklendirme Kuralı

C++ programlama dilinde **değişkenlerin kimliklendirilmesinde (identifier definition)** dört temel kurallar vardır. Bunlar;

1. **Kimliklendirmede anahtar kelimeler kullanılamaz!**
2. **Kimliklendirme asla rakamla başlayamaz!**
3. **Kimlikler en fazla 32 karakter olmalıdır!** Daha fazlası derleyici tarafından dikkate alınmaz!
4. **Kimliklendirmede ancak İngiliz alfabesindeki Büyük ve Küçük harfler, rakamlar ile alt çizgi karakteri kullanılabilir.**

Aşağıda kimliklendirme kurallarına uygun tanımlama yapılmıştır;

```
/* Bu program değişken tanımlama örneklerini içerir. */
int main() {
    /* Tam sayı değişkenler: */
    char yas;
    short kisa_tam_sayi;
    long uzun_tam_sayi;
    /* Gerçek Sayı Değişkenler:*/
    float kilo;
    double reel_sayi;
    /* Aynı veri tipinde birden fazla değişkene aralarına virgül koyarak kimlik verilebilir. */
    int kat1,kat2,kat3;
    float olcek1,olcek2;
}
```

Bütün bu kuralların yanında değişkenlere kimlik verilirken yazılan kodun okunaklılığını artırmak için **deve yazımı (camel case)** stili kimlik verilir. Bu stilde ilk sözcük tamamıyla küçük, izleyen sözcüklerin ise baş harfi büyük yazılır. Bu kural zorunlu değildir ama koda bakım yapacak sonraki programcıların işini kolaylaştırmak için ahlaki olarak tercih edilir;

```
/* Bu program camelCase değişken kimliklendirme örneklerini içerir.*/
int main() {
```

```

int sayac;
int ogrenciBoyu;
int asansorunOlduguKat;
float asansorAgirligi;
int ogrenciYasi;
char ilkHarf;
char ogrenciAdi[50];
char ogrenciSoyadi[50];
float ortalamaNot;
}

```

Değişkenin kapsamı (**variable scope**); Değişkenin bildiriminin (**variable declaration**) yapılabileceği, değişkene erişilebileceği (**access**), değişkenin üzerinde çalışılabileceği program kodu olarak tanımlanır.

Değişken Kapsamı

Değişkenin kapsamı, **tanımlandığı kod bloğu (ve varsa bu blok içindeki iç bloklar)** ile sınırlıdır. Sınıflar ve türemiş sınıflar için bunun istisnası vardır. İleride bahsedilecektir.

§4.3.6 Veri Tipi Değiştiricileri

İşaretsiz bir tam sayı kullanmak yerine en anlamlı işaret bitini de sayıya dahil ederek tam sayıları **işaretsiz** olarak (**unsigned**) kullanabiliriz. Tam sayı veri tipleri olan **char**, **short**, **int**, **long** ve **long long** veri tipleri (**data type**) aslında **işaretsiz** (**signed**) tam sayılardır. Yani önlerinde **signed** sıfatı olduğu varsayılır. Tam sayıları işaretsiz olarak kullanmak istememiz halinde **unsigned** sıfatını tam sayı veri tipinin önünde kullanırız.

```

/* Bu program işaretsiz tam sayı değişken tanımlamak için oluşturulmuştur.*/
int main() {
    unsigned yas; /* veri tipi "unsigned char" ve kimliği "yas" değişkeni
tanımlandı. yas değişkeni 0 ile 255 arasında bir değer alabilir. */
    unsigned short kat; /* veri tipi "unsigned short" ve kimliği "kat" olan değişken
    ↳ tanımlandı.kat değişkeni 0 ila 65535 arasında değer alabilen bir değişkendir. */
    unsigned long pozitifTamsayi; /* pozitifTamsayi değişkeni 0 ile 4.294.967.295 arasında değer
    ↳ alabilen bir değişkendir. */
    uint8_t ogrenciYasi; //işaretsiz veri tipi kullanarak yaş tanımlı.
}

```

Yukarıda 4.3 başlığında verilen temel veri tiplerini değiştiren **unsigned** gibi **veri tipi değiştiricileri** (**data type modifier**) vardır. Değiştiriciler için kullanılan anahtar sözcükler;

- ◊ Bir değişkenin değerinin ilk değer atamasından sonra değiştirilemeyeceğini **const** sıfatı belirtir.
- ◊ Bir veri tipinin hem pozitif hem de negatif değerleri tutabileceğini **signed** sıfatı belirtir. Örneğin 8 bitlik **char** veri tipinin en anlamlı olan 8. bitini işaret biti olarak kullanılmasını sağlar. Böylece -127 ile +128 arasındaki sayılar **signed char** olarak tanımlanabilir.
- ◊ Bir veri tipinin yalnızca negatif olmayan değerleri tutabileceğini **unsigned** sıfatı belirtir. Örneğin 8 bitlik **char** veri tipinin en anlamlı olan 8. bitini sayı olarak kullanılmasını sağlar. Böylece 0 ile 255 arasındaki sayılar **unsigned char** olarak tanımlanabilir.
- ◊ Tam sayı tipinin daha kısa bir sürümünü **short** belirtir. Yani 16 bit tam sayı tanımlamak için **short int** veri tipi kullanılabilir.
- ◊ Tam sayı tipinin daha uzun bir sürümünü **long** belirtir. Yani 32 bit tam sayı tanımlamak için **long int** veri tipi kullanılabilir.
- ◊ Bir değişkenin değerinin programcının hazırladığı mevcut kod dışında beklenmedik şekilde değişebileceğini ve derleyicinin bazı optimizasyonları yapmaması gerektiğini **volatile** sıfatı belirtir. Örnek olarak bir aygıt tarafından veya bir **kesme** (**interrupt**) tarafından bir değişken değiştirilebilir. Derleyiciye bir değişkenin değerinin kodu yazılmış program tarafından değil de başka şekillerde değiştirilebileceğini söyler.

```

/* Bu program işaretsiz tam sayı değişken tanımlamak için oluşturulmuştur.*/
int main() {
    unsigned char ogrenciYasi; /* veri tipi "unsigned char" ve kimliği "ogrenciYasi" olan
değişken bildirimi. Bu değişken 0 ile 255 arasında bir değer alabilir. */
    unsigned short asansorunBulunduguKat; /* veri tipi "unsigned short" ve kimliği
"asansorunBulunduguKat" olan değişken bildirimi. Bu değişken, 0 ile 65.535 arasında
değer alabilen bir değişkendir */
}

```



```

unsigned long int pozitifTamsayi; /* pozitifTamsayi değişkeni 0 ile 4.294.967.295 arasında
→ değer alabilen bir değişkendir. */
volatile int deviceStatus=0; /* Programcının yazacağı kod dışında bu değişken
→ değiştirilebilir! */
}

```

Sabit Bit Sayısına Sahip Veri Tipleri

Profesyonel yazılım mimarisi ve taşınabilir kod için, sabit bit sayısına sahip işaretli veri tiplerinin (`uint8_t`, `uint16_t`, `uint32_t`, `uint64_t`) kullanılması şiddetle tavsiye edilir. Tam sayı veri tipleri 4.2 başlığında bu tiplere yer verilmiştir. Bu tipler, tüm platformlarda tam olarak doğru bit sayısını garanti eder.

4.4 Değişmezler

Programcı, kodlama yaparken bazı **sabitleri** (**constant**) ister istemez kullanır. Sabitler iki türlü tanımlanır. Bunların biri **değişmez** (**literal**), diğeri ise değeri değiştirilemeyen **sabit değişkenlerdir** (**const variable**). Derleme öncesinde de sonrasında da kaynak koddakinin kelimesi kelimesine aynısı olan **değişmezler** (**literal**) kullanılır. Değişkenlere verilen **ilk değerler** (**initial value**) ile **katsayılar** (**coefficient**) en çok kullanılan değişmezlerdir.

Başlatma

Değişkenler tanımlanırken sahip olacağı ilk değerleri değişmezler ile verme işlemine **başlatma** (**initialization**) adı verilir.

Aşağıda çeşitli değişkenleri başlatma örneği verilmiştir;

```

1  /* Bu program değişkenleri değişmez (literal) ile başlatma örneklerini içerir. */
2  int main() {
3      short i,j=0;
4      char c=65;
5      float f=2.5f;
6      char harf='A'; //char harf=65;
7      bool dogruMu=false;
8  }

```

Yukarıda verilen kod şu şekilde açıklanabilir;

- ◇ 3. Satırda; `short` veri tipindeki `i` değişkeni tanımlandı ve yine `short` veri tipindeki `j` kimlikli değişkene 0 tam sayı değişmezi (**integer literal**) ile başlatıldı.
- ◇ 4. Satırda; `char` veri tipindeki `c` kimlikli değişkende 65 tam sayı değişmezi ile başlatılmıştır.
- ◇ 5. Satırda; `float` veri tipindeki `f` kimlikli değişkene Türkçe yazılışı 2,5 olan kayan noktalı değişmezi (`f` harfi tek incelikli olduğunu ifade eder) ile başlatıldı. Kod İngilizce yazıldığından, kodun içerisindeki gerçek sayı değişmezleri için **ondalık ayracı nokta olarak yazılır**.
- ◇ 6. Satırda; `char` veri tipindeki `harf` kimlikli değişkene 'A' karakter değişmezi (**character literal**) ile başlatıldı. **Karakter değişmezler tek tırnak içinde yazılır**. Klavyeden basılan her bir harf bellekte sayı olarak tutulur. 'A' karakterinin kodu 65, 'B' karakterinin kodu 66, ... Aşağıda detaylı açıklama verilmiştir.
- ◇ 7. Satırda; `bool` veri tipindeki `dogruMu` kimlikli değişkene `false` bool değişmezi (**bool literal**) ile başlatıldı.

Karakter kodları için genellikle ASCII (**American Standard Code for Information Interchange**) kümesi kullanılır ve 0 ile 127 arasındaki karakterleri içeren 7 bitlik bir karakter kümesidir. ANSI (**American National Standards Institute**) kümesi ise, 8 bitlik karakter kümeleri için kullanılır. Bu karakter kümeleri, değişmemiş ASCII karakter kümesini içerir. ANSI karakter kümesi, ek olarak, 128 ila 255 arasında ek karakterler içerir. Bunlar Batı Avrupa özel karakterleri ile Arapça, Yunanca veya Kiril karakterleri gibi karakterlerdir.

Boş Karakter

Kodu 0 olan karakter ise **boş karakter** (**NULL Character**) olarak adlandırılır.

Aşağıda birçok veri tipi için değişmez (**literal**) örneği tablo olarak verilmiştir. Bunların birçoğu ilerleyen konularda anlatılacaktır. Bu tabloda değişmezler tanımlanırken her zaman büyük harf son ekleri (örneğin `L` veya `LL`) tercih edilir. Küçük `l`, bazı fontlarda bir rakamı ile karıştırılabildiği için modern yazım standartlarında

Kategori	Veri Tipi	Değişmez Örneği	Açıklama/Önek/Sonek
Tam Sayı	int	42	Varsayılan (onluk)
	unsigned int	42u veya 42U	u veya U soneki
	long	42l veya 42L	l veya L soneki
	unsigned long	42ul veya 42UL	ul soneki
	long long	42ll veya 42LL	ll veya LL (64-bit)
	İkili (binary)	0b101010	0b ön eki (C++14)
	Onaltılı (hexadecimal)	0x2A	0x ön eki
Kayan Noktalı	double	3.1415	Varsayılan (8 byte)
	float	3.14f veya 3.14F	f veya F (4 byte)
	long double	3.14l veya 3.14L	l veya L (12-16 byte)
Karakter	char	'A' veya "ABC"	Standart (UTF-8/ASCII)
	wchar_t	L'A' veya L"ABC"	L ön eki (Geniş Karakter)
	char8_t	u8"ABC"	u8 öneki (C++20)
	char16_t	u'A' veya u"ABC"	u ön eki (UTF-16)
	char32_t	U'A' veya U"ABC"	U ön eki (UTF-32)
Mantıksal	bool	true, false	Doğru veya Yanlış
Gösterici	std::nullptr_t	nullptr	Boş gösterici (C++11)

Tablo 4.4: Tüm Veri Tipleri İçin Değişmez Örnekleri

büyük harf daha güvenlidir. Ayrıca ileride göreceğimiz, C++11 ile gelen **kullanıcı tanımlı değişmez UDL** (**user defined literal**) işleci (**operator**) sayesinde `100.0_km` gibi fiziksel birimleri koda ekleyebiliriz.

Aşağıdaki kod örneğinde programcı, sırasıyla `3`, `3.14f`, `2.0f` ve `3.14f` olmak üzere dört farklı değişmez (**literal**) kullanılmıştır. Bu değişmezlerin her biri derleyici tarafından farklı olarak değerlendirilir ve çalışma anında (**run-time**) değiştirilemez.

```
/* Bu program, değişmez örneğidir. */
int main() {
    int yariCap=3;
    float daireninAlani=3.14f*yariCap*yariCap;
    float daireninCevresi=2.0f*3.14f*yariCap;
}
```

Değişkenler tanımlanırken parantez içerisinde değişkeler verilerek de başlatılabilir. Aşağıda yeni nesil başlatma (**initialize**) örnekleri verilmiştir;

```
/* Bu program değişkenlere C++ tipi ilk değer verme ve değişmez örneklerini içerir. */
int main() {
    int tamsayi(61); /* "tamsayi" kimlikli "int" veri tipindeki değişken tanımlandı.
ve ilk değeri 61 değişmezi ile verildi */
    char enter(13), tab(8), buyukA('A'); /* "enter" kimlikli "char" veri tipindeki
değişkene 13 değişmezi ile ilk değer verildi. "tab" kimlikli "char" veri tipindeki
değişkene 8 değişmezi ile ilk değer verildi. "buyukA" kimlikli "char" veri tipindeki
değişkene 'A' karakter değişmezi ile ilk değer verildi. */
    double reel(2.5); /* "reel" kimlikli "double" veri tipindeki değişkene Türkçe 2,5
değişmezi ile ilk değer verildi. Ondalık ayracı NOKTA olarak yazılır. */
    bool test(false); /* "test" kimlikli "bool" veri tipindeki değişkene
false değişmezi ile ilk değer verildi. false yerine 0 true yerine 1 de yazılabilir.*/
}
```

4.5 Sabit Değişkenler

Değişmezlerin (**literal**) çokça kullanılması derleme sonrası üretilen icra edilebilir makine kodunu da büyütür. Bunun yerine çok kullanılan değişmezler bellekte bir kez yer kaplasın diye **const sıfatıyla** (**const qualifier**) **sabit değişkenler** (**const variable**) tanımlanabilir.

Sabit Değişkenler

Sabit değişkenler etik olarak büyük harfle kimliklendirilir. Sabit değişkenler, daha sonradan değiştirilemeyeceğinden **başlatılmalıdır** (**initialize**)!

```
/* Bu program, const sıfatı örneğidir. */
```



```
int main() {
    int yariCap=3;
    const float PI(3.14f); // Sabit değişken tanımı!
    float daireninAlani=PI*yariCap*yariCap;
    float daireninCevresi=2.0f*PI*yariCap;
}
```

Böylece aynı sabit `PI` sabit değişkeni birden çok yerde kullanılır ve her seferinde aynı bellek bölgesine erişildiğinden bellekten tasarruf edilmiş olunur. Tanımlama (**definition**) sırasında `const` sıfatı kullanılan değişkene, verilen ilk değer (**initial value**) değişmez (**literal**) olarak verilmelidir. **Daha sonra bu değişkene bir değer ataması olamaz!**

Bir değişkeni sabit tanımlamanın üstünlükleri aşağıda sıralanmıştır;

- ♦ **Gelişmiş Kod Okunabilirliği:** Bir değişkene `const` sıfatı vermek, diğer programcılara değerinin değiştirilmemesi gerektiğini belirtir; bu da kodun anlaşılmasını ve sürdürülmesini kolaylaştırır.
- ♦ **Gelişmiş Veri Tipi Güvenliği:** `const` kullanılması, değerlerin yanlışlıkla değiştirilmemesini sağlayabilir ve kodumuzda hata ve eksiklik olma olasılığını azaltabilir.
- ♦ **Gelişmiş Optimizasyon:** Derleyiciler, değerlerinin program yürütme sırasında değişmeyeceğini bildikleri için sabit değişkenlerini daha etkili bir şekilde optimize edebilirler. Bu, daha hızlı ve daha verimli kodla sonuçlanabilir.
- ♦ **Daha İyi Bellek Kullanımı:** Değişkenlere `const` sıfatı vermek, değerlerinin bir kopyasını oluşturmayı engeller; bu da bellek kullanımını azaltabilir ve performansı artırabilir.
- ♦ **Gelişmiş Uyumluluk:** Değişkenlere `const` sıfatı vermek, kodunuzu sabit değişkenleri kullanan diğer başlıklarla daha uyumlu hale getirebilir.
- ♦ **Gelişmiş Güvenilirlik:** `const` kullanarak, değerlerin beklenmedik şekilde değiştirilmemesini sağlayarak kodunuzu daha güvenilir hale getirebilir ve kodunuzdaki hata ve eksiklik riskini azaltabilirsiniz.

4.6 Sabit İfadeler

C++'ta sabit değişkenler gibi bir de **sabit ifadeler** (**constant expression**) vardır.

Sabit İfadeler

Sabit ifadeler `constexpr` anahtar kelimesiyle tanımlanır. Bir değişkenin veya fonksiyonun değerinin **derleme zamanında** (**compile-time**) hesaplanması gerektiğini belirten bir anahtar kelimedir.

C++11 ile gelmiş ve modern C++'ın en güçlü özelliklerinden biri olmuştur. Temel mantığı şudur: "Eğer bir hesaplamayı program çalışırken değil de derlenirken yapabiliyorsak, yapalım; Böylece program daha hızlı çalışır." Sabit ifadelerin hesaplaması, derleme sırasında yapıp sonuç doğrudan koda gömüldüğü için, çalışma anında (**run-time**) işlemci yorulmaz ve icra hızlanır. Değer derleme aşamasında belirlendiği için, hatalı bir durum varsa (örneğin sifıra bölme) derleyici bunu size hemen söyler. C++'ta standart dizilerin boyutu sabit olmalıdır. `constexpr` bir değeri dizi boyutu olarak güvenle kullanabilirsiniz.

Özellik	<code>const</code>	<code>constexpr</code>
Zamanlama	Çalışma zamanında (run-time) belirlenebilir.	Sadece derleme zamanında (compile-time) belirlenir.
Başlatma	Değişkenin değeri çalışma anında (run-time) (örnek olarak kullanıcı girişi ile) atanabilir.	Değer mutlaka derleme anında (compile-time) bilinen bir sabit olmalıdır.
Temel Amaç	Verinin değiştirilmesini engellemek (read-only).	Performans artışı ve derleme zamanı hesaplamaları.
Dizi Boyutu	Her zaman dizi boyutu olarak kullanılamaz (derleyiciye bağlı).	Güvenle dizi boyutu (array size) olarak kullanılabilir.
Fonksiyonlar	Fonksiyon dönüş değerleri her zaman sabit değildir.	Fonksiyonun sonucu derleme anında hesaplanabilir.

Tablo 4.5: C++ `const` ve `constexpr` Karşılaştırması

Aşağıda verilen örnek programı inceleyiniz;

```
int x;
std::cin >> x;
const int sabit1 = x; // DOĞRU: x çalışma anında belli olur.
// constexpr int sabit2 = x; // HATA: x derleme anında bilinmiyor!
```

`constexpr int` `sabit3 = 10 * 5;` // DOĞRU: Derleyici sonucu 50 olarak hesaplayıp koyar.

4.7 Yazdığımız Kod Nasıl İcra Edilir?

Derleyici tarafından işlemcinin anlayacağı makine diline çevrilen programımız aslında **yazdığımız talimatları (statement) sırasıyla çalıştırır**. C++ dilinde her talimat noktalı virgül ile bittiği 4.2.2 başlığında anlatılmıştı. Aşağıdaki örnek programı göz önüne alalım;

```
1 int main() {
2     float boy(1.80);
3     float kilo(100.0);
4     float bki;
5     float boyKare;
6     boyKare=boy*boy;
7     bki=kilo/boyKare;
8     boy=1.50;
9 }
```

Yapısal programlamada program, ana fonksiyondan başlar. Bu nedenle icra, `main()` fonksiyonu içindeki ilk talimatla başlar ve aşağıdaki sırayla icra edilir;

1. İcra edilecek ilk talimat, 2. satırdaki `float boy(1.80);` talimatıdır. `boy` Değişkenine bellek tahsis edilerek değişkene `1.80` değeri atanır. Yani `boy` değişkeni `1.80` değeriyle **başlatılır (initialize)**.
2. Sonra 3. satırdaki `float kilo(100.0)` talimatı icra edilir. `kilo` Değişkeni `100.0` ile başlatılır.
3. Daha sonra 4. satırdaki `float bki;` talimatı icra edilir. Tanımlanan `bki` değişkenine sadece bellek tahsis edilir ve değeri belli değildir. Bazı derleyiciler değerleri sıfır olarak varsayar.
4. Daha sonra 5. satırdaki `float boyKare;` talimatı icra edilir. Tanımlanan `boyKare` değişkenine sadece bellek tahsis edilir ve değeri belli değildir.
5. Daha sonra 6. satırdaki `boyKare=boy*boy;` icra edilir. Önceden tanımlanmış `boyKare` değişkenine `boy` değişkeni kendisiyle çarpılarak işlem sonucu atanır. `boyKare` değişkeni öncesinde tanımlanmamış olsaydı derleme hatası alacaktık.
6. Daha sonra 7. satırdaki `bki=kilo/boyKare;` talimatı icra edilir. `kilo` Değişkenini `boyKare` değişkenine böler ve çıkan sonucu `bki` değişkenine atar.
7. Son olarak 8. satırdaki `boy=1.50` talimatı icra edilir. `boy` Değişkenine `1.50` atanarak değeri değiştirilir.

4.8 İşleçler

İşleçler (operator) matematikten bildiğimiz işleçlerdir.

$$y = 3 + 2$$

Yukarıda verilen cebirsel ifadede toplama işleci (+) iki tarafına bulunan argümanları toplar. Bu argümanlara **işlenen (operand)** adı verilir. Yüksek düzey dillerde de aritmetik işleçlerin yanında birçok işleç de bulunur.

Atama İşleci

En çok kullanılan işleç, atama (=) işlecidir. **Atama işleci, sağdaki işleneni soldaki değişkene atar**. Bu nedenle kodlama yapılırken `2+2=x` veya `a+b=x` yazılamaz!

§4.8.1 Aritmetik İşleçler

C++ programlama dilinde, C dilinde olduğu gibi, aritmetik işleçlerin çalışma şekli **işlenenlerin (operand)** veri tipine göre değişir. Aritmetik işlemler ve atama işleci için kurallar;

1. **İşlenenlerden birinin bellekte kapladığı yer küçük ise ikisi aynı olacak şekilde bellekte daha fazla yer kaplayanına dönüştürülür** ve işlem sonra yapılır ve sonuç dönüştürülen veri tipinde üretilir. Örnek olarak `char` veri tipindeki değişken ile `long` veri tipindeki bir değişken toplanmaya çalışıldığı zaman `char` veri tipindeki değişkenin değeri `long` veri tipine otomatik dönüştürülür. Sonuç `long` veri tipinde üretilir.
2. **İşlenenlerden biri tam sayı diğeri kayan noktalı sayı ise işlem yapılmadan önce işlenenler kayan noktalı sayıya dönüştürülür**. Örneğin `int` veri tipindeki değişken ile `float` veri tipindeki bir değişken toplanmaya çalışıldığı zaman `int` veri tipindeki değişkenin değeri `float` veri tipine otomatik dönüştürülür. Sonuç `float` veri tipinde üretilir.

Aritmetik işleçlerle (arithmetic operator), temel dört işlem ile modül işlemleri yapılır.

İşleç	Adı	Açıklama
+	Toplama	İki işleneni toplar.
-	Çıkarma	Soldaki işlenenden sağdakini çıkarır.
*	Çarpma	İki işleneni çarpar.
/	Bölme	Soldaki işleneni sağdakine böler. Her iki işlenen tam sayı ise sonuç tam sayıdır (ondalık kısım atılır).
%	Modül	Soldaki işlenenin sağdakine bölümünden kalan değeri verir.

Tablo 4.6: Aritmetik İşleçler

Aritmetik işleçlere (arithmetic operator) ilişkin örneği aşağıdaki görebilirsiniz;

```
/* Bu program, aritmetik işleç örneğidir. */
int main() {
    int a = 9, b = 4, res;
    float fa=6.6, fb= 2.2, fres;
    res = a + b; // işlem sonucu res=13
    res = a - b; // işlem sonucu res=5
    res = a * b; // işlem sonucu res=36
    res = a / b; // işlem sonucu res=9/4=2 tam 1/4=2
    /* Bölme işlecinin işlenenleri tam sayı olduğunda bölme sonucundaki tam sayı olarak
       ↪ hesaplanır. Bölme işleminin tam kısmını içeren sonuç döndürülür. */
    res = a / 4.3 ; /* res tam sayı olduğundan işlem sonucu; res=9/4.3 =9.0/4.3 =2.0930 =2 tam
       ↪ 0.0930 =2 */
    res = a % b; // işlem sonucu res=9%4= 2 tam 1/4=1
    fres= fa / fb; // işlem sonucu res=6.6/2.2=3.00
}
```

C++ programlama dilinde tam sayı bölme işlemi işlemcinin en basit yetenekleriyle çözülür. Bu nedenle diğer programlama dillerinden ayrılır. Aşağıda buna ilişkin örnek verilmiştir.

```
/* Bu program, aritmetik işleç BÖLME örneğidir. */
int main () {
    int a = 19 / 2 ; // a = 2*9+1 = 9
    int b = 18 / 2 ; // b = 9
    int c = 255 / 4; // c = 4*63+3 = 63
    int d = 45 / 4 ; // d = 4*11+1 = 11
    double aa = 19 / 2.0 ; // aa = 9.5
    double bb = 18.0 / 2 ; // bb = 9.0
    double cc = 255 / 2.0; // cc = 127.5
    double dd = 45.0 / 4 ; // dd = 11.25
    double ee = 45 / 4 ; /* ee = 11 çünkü bölme tam sayılar arasında yapılmıştır */
}
```

§4.8.2 Tekli İşleçler

Tekli işleçler (unary operator) sadece bir işlenen üzerinde işlem yapar. Tekli demek tek bir işlenen (**operand**) ile işlem yapması anlamındadır.

İşleç	Adı	Açıklama
+	Tekli Artı	İşlenenin işaretini pozitif yapar (genellikle varsayılan durumdur).
-	Tekli Eksi	İşlenenin işaretini tersine çevirir (x ise $-x$, $-x$ ise x yapar).
++	Arttırım	İşlenenin değerini 1 artırır. Önek (prefix) ise önce artırır sonra değeri döndürür. Sonek (postfix) ise önce değeri döndürür sonra artırır.
--	Eksiltme	İşlenenin değerini 1 eksiltir. Önek ve sonek kullanım kuralları artırım işleci ile aynıdır.
!	Mantıksal Değil	bool değerini tersini alır (true ise false yapar).
(type)	Tip Dönüşümü	İşleneni parantez içinde belirtilen veri tipine (casting) dönüştürür.
&	Adres Operatörü	Değişkenin bellek adresini döndürür.
*	Başvuru Kaldırma	Göstericinin (pointer) işaret ettiği değeri döndürür.
sizeof	Bellek Boyutu	sizeof temel veri tiplerin bellek boyutunu geri döndürür.

Tablo 4.7: Tekli İşleçler

Aşağıda tekli işleçlerin kullanımına ilişkin örnek program verilmiştir;

```
/* Bu program, tekli işleç örneğidir. */
int main() {
    int a = 5, b = 5, res;
    // Tekli eksi (unary minus)
    res = -a;
    // Önce-artırım (Pre-increment)
    res = ++a + 2;
    /* a önce 6 olur, sonra 2 ile toplanır. res=8, a=6 */
    // Sonra-artırım (Post-increment)
    res = 2 + b++;
    /* İşlem b=5 ile yapılır (res=7), sonra b 6 olur. */
    // Değerleri sıfırlayalım
    a = 5; b = 5;
    // Önce-azaltım (Pre-decrement)
    res = --a + 2;
    /* a önce 4 olur, sonra 2 eklenir. res=6, a=4 */
    // Sonra-azaltım (Post-decrement)
    res = 2 + b--;
    /* İşlem b=5 ile yapılır (res=7), sonra b 4 olur. */
    // Mantıksal DEĞİL (Logical NOT)
    //C Dilinden gelen
    a = 1; b = 0;
    res = !a; // res = 0 (false)
    res = !b; // res = 1 (true)
    //C++ yöntemiyle
    bool ba=true, bb=false, bres;
    bres = !ba; // res = false
    bres = !bb; // res = true
    // sizeof işleci
    res = sizeof a; //4
    res = sizeof(char); //1
}
```

§4.8.3 İlişkisel İşleçler

İlişkisel işleçler (**relational operator**), işlenenlerin arasındaki ilişkiyi verirler. C++ dilinde, doğru (**true**) ya da yanlış (**false**) olabilen **bool** veri tipinde değer döndürürler.

İşleç	Adı	Açıklama
==	Eşittir	Sol ve sağdaki işlenenler birbirine eşitse true döner.
!=	Eşit Değildir	İşlenenler birbirine eşit değilse true döner.
>	Büyüktür	Soldaki işlenen sağdakinden büyükse true döner.
<	Küçüktür	Soldaki işlenen sağdakinden küçükse true döner.
>=	Büyük veya Eşit	Soldaki işlenen sağdakinden büyük veya ona eşitse true döner.
<=	Küçük veya Eşit	Soldaki işlenen sağdakinden küçük veya ona eşitse true döner.
<=>	uzay gemisi (spaceship operator)	C++20 ile hayatımıza girmiştir. Bu operatör tek bir işlemle küçük-tür, eşittir veya büyüktür durumunu döndürür. İleride anlatılacaktır.

Tablo 4.8: İlişkisel İşleçler ve Mantıksal Karşılıkları

Aşağıda verilen ilişkisel işleçlerin kullanımına ilişkin örnek programı lütfen inceleyiniz.

```
/* Bu program, ilişkisel işleç örneğidir. */
int main() {
    int a = 5, b = 6;
    bool res;
    res = a<b; // işlem sonucu res=true
    res = a<=b; // işlem sonucu res=true
    res = a>b; // işlem sonucu res=false
    res = a>=b; // işlem sonucu res=false
    res = a==b; // işlem sonucu res=false
}
```

```

    res = a!=b; // işlem sonucu res=true
}

```

§4.8.4 Bit Düzeyi İşleçler

Bit düzeyi işleçler (bitwise operator) özellikle ikilik tabandaki tam sayılarda karşılıklı bitler üzerinde yapılan işlemlerdir. Aslında bu işlemler çarpma, toplama, bölme ve çıkarma işlemleri için kullanılır.

İşleç	Adı	Açıklama
&	VE (AND)	Karşılıklı iki bit de 1 ise sonuç 1, diğer durumlarda 0 olur.
	VEYA (OR)	Karşılıklı bitlerden en az biri 1 ise sonuç 1, ikisi de 0 ise 0 olur.
^	ÖZEL VEYA (XOR)	Bitler birbirinden farklıysa 1, aynıysa 0 olur.
~	DEĞİL (NOT)	Tüm bitleri tersine çevirir (0 → 1, 1 → 0).
<<	Sola Kaydırma	Bitleri belirtilen sayı kadar sola kaydırır. Sağdan 0 ile doldurur.
>>	Sağa Kaydırma	Bitleri belirtilen sayı kadar sağa kaydırır.

Tablo 4.9: Bit Düzeyi İşleçler

Aşağıda tam sayılar üzerinde yapılan bit düzeyi işlemler gösterilmiştir;

```

/* Bu program, bit düzeyi işleç örneğidir. */
int main() {
    char a = 00000110; // 6
    char b = 00000010; // 2
    char bitwise_and = a & b; // 00000010
    char bitwise_or = a | b; // 00000110
    char bitwise_xor = a ^ b; // 00000100
    char bitwise_not = ~b; // 00001101
    char left_shift = b << 2; // 00000100
    char right_shift = b >> 1; // 00000001
}

```

§4.8.5 Mantıksal İşleçler

Mantıksal ifadeler doğru (**true**) ve yanlış (**false**) olabilen ifadelerdir. C++ dilinde mantıksal bir veri tipi **bool** veri tipidir. C++ Dilinde VE(AND), VEYA(OR) ve DEĞİL (NOT) gibi mantıksal ifadelerin işlenmesi için aşağıdaki işleçler kullanılır. Mantıksal işleçlere (**logical operator**) işlenen olarak giren ifadeler öncelikle **bool** veri tipine dönüştürülür. Yani bu işleçler için beklenen işlenen (**operand**) bir tamsayı ifadedir.

İşleç	Mantıksal İşlem ve Kısa Devre Davranışı
&&	Koşullu Mantıksal VE: Her iki işlenen de 1 ise 1 döner. Soldaki işlenen 0 ise sağdaki ifade değerlendirilmez (Kısa Devre).
	Koşullu Mantıksal VEYA: İşlenenlerden en az biri 1 ise 1 döner. Soldaki işlenen 1 ise sağdaki ifade değerlendirilmez (Kısa Devre).
!	Mantıksal DEĞİL: İşlenenin mantıksal durumunu tersine çevirir. 1 değerini 0, 0 değerini 1 yapar.

Tablo 4.10: Mantıksal İşleçler ve Kısa Devre Özelliği

Aşağıda verilen ilişkisel ve mantıksal işleçlerin kullanımına ilişkin örnek programı lütfen inceleyiniz.

```

/* Bu program, mantıksal işleç örneğidir. */
int main() {
    int a = 5, b = 6;
    bool res;
    res = a!=5 && b==6; // işlem sonucu res=0
    res = a!=5 || b==6; // işlem sonucu res=1
    res = !a!=5; // işlem sonucu res=1
    return 0;
}

```

Bu işleçlerin çalışma şekli doğruluk tablosundaki gibidir ancak derleyici optimizasyonu gereği, ifadenin bir kısmı işlenmediğinden, **koşullu olarak çalışırlar**. Buna **kısa devre** adı verilir. Derleyici, sonucun kesinleştiği noktada işlem yapmayı bırakır. Örneğin (**a == 5 && b == 10**) ifadesinde **a** değeri 5 değilse, **b**'nin ne olduğuna bakmaya gerek duymaz. Bu durum sadece hız kazandırmaz, aynı zamanda olası hataları da önler:

```
int a=10, b=10;
```

```
if (a == 5 && b == 10) // a 5 den farklı olduğu için b==10 kontrol edilmez!
{
    // Buradaki işlemler yapılmaz!...
}
```

C dilinde tekli & ve | işleçleri de mantıksal ifadelerle kullanılabilir. Ancak bunlar **kısa devre yapmazlar**; yani sol taraf ne olursa olsun sağ tarafı da kontrol ederler.

§4.8.6 Atama işleçleri

Atama işleçleri (**assignment operator**) en çok kullanılan işleçlerdir.

Atama İşleçleri

Atama işleçleri her zaman sağdaki değeri sola atar! Dolayısıyla soldaki işlenen, atama yapılacak uygun veri tipinde bir değişken (**variable**) olmak zorundadır!

İşleç	Adı	Örnek	Açıklama
=	Temel Atama	x = 5	Sağdaki değeri soldaki değişkene atar.
+=	Toplayarak Atama	x += 3	x = x + 3 ile aynıdır.
-=	Çıkararak Atama	x -= 2	x = x - 2 ile aynıdır.
*=	Çarparak Atama	x *= 4	x = x * 4 ile aynıdır.
/=	Bölerek Atama	x /= 2	x = x / 2 ile aynıdır.
%=	Modül Alarak Atama	x %= 3	x = x % 3 ile aynıdır.
&=	Bit Düzeyi VE ile Atama	x &= 1	x = x & 1 ile aynıdır.
=	Bit Düzeyi VEYA ile Atama	x = 1	x = x 1 ile aynıdır.
^=	Bit Düzeyi ÖZEL VEYA ile Atama	x ^= 1	x = x ^ 1 ile aynıdır.
<<=	Bit Düzeyi Sola Kaydırma ile Atama	x <<= 1	x = x << 1 ile aynıdır.
>>=	Bit Düzeyi Sağa Kaydırma ile Atama	x >>= 1	x = x >> 1 ile aynıdır.

Tablo 4.11: Atama İşleçleri

Aşağıda verilen atama işleçlerin kullanımına ilişkin örnek programı lütfen inceleyiniz.

```
/* Bu program, atama işleçleri örneğidir. */
int main() {
    int a = 10; // a değişkenine 10 değışmezi = işleciyle atanıyor.
    a += 10; // a=a+10; ile eşdeğer: a değışkenine 10 ekleniyor.
    a -= 10; // a=a-10; ile eşdeğer: a değışkeninden 10 çıkarılıyor.
    a *= 10; // a=a*10; ile eşdeğer: a değışkeni 10 kat yapılıyor.
    a /= 10; // a=a/10; ile eşdeğer: a değışkeni 1/10 kat yapılıyor.
    a %= 10; // a=a%10; ile eşdeğer: a değışkeninin 10 a kalanı bulunup ona eşitleniyor.
}
```

§4.8.7 İşleç Öncelikleri

Cebirsel ifadelerde nasıl ki önce çarpma ve bölme sonra toplama işlemleri yapılıyor. C programlama dilinde yazılan ifadelerde de işleç öncelikleri (**operator precedence**) vardır. İfadelerde bir işlemi öncelikli hale getirmek için parantez içerisine alınır. En içteki parantez en öncelikli olarak gerçekleştirilir (Tablo 4.12).

Başka işleçler de bulunmaktadır bunlar ilerleyen bölümlerde yeri geldikçe anlatılacaktır. Aşağıda işleçlerin önceliklerine ilişkin örnek programı lütfen inceleyiniz.

```
/* Bu program, işleç öncelikleri örneğidir. */
int main() {
    int a=10, b=5;
    a = a-b+2; // Öncelik Sırası +, -, = İşlem sonunda a=7
    a = a/b+2; // Öncelik Sırası /, +, = İşlem Sonunda a=7/5+2=3
    a = 2*10-20/2+3+(12-10); // Öncelik Sırası: (-) * / - + +
    /* İfadenin çözüm sırası:
    > 2*10-20/2+3+2
    > 20-20/2+3+2
    > 20-10+3+2
    > 10+3+2
    > 13+2
```



```
> 15 */
}
```

Öncelik	Kategori	İşleçler	Birleşme
1	Kapsam	::	Soldan Sağa
2	Temel (postfix)	f(), [], -, ++, x-, typeid, const_cast...	Soldan Sağa
3	Tekli (unary)	++, -, *, ~, *, &, sizeof, new, delete, alignof	Sağdan Sola
4	Üye Seçimi	.*, ->*	Soldan Sağa
5	Çarpma/Bölme	*, /, %	Soldan Sağa
6	Toplama/Çıkarma	+, -	Soldan Sağa
7	Bit Kaydırma	<<, >>	Soldan Sağa
8	Üç Yollu Karş.	<=> (C++20)	Soldan Sağa
9	İlişkisel	<, <=, >, >=	Soldan Sağa
10	Eşitlik	==, !=	Soldan Sağa
11	Bit Düzeyi VE	&	Soldan Sağa
12	Bit Düzeyi XOR	^	Soldan Sağa
13	Bit Düzeyi VEYA		Soldan Sağa
14	Mantıksal VE	&&	Soldan Sağa
15	Mantıksal VEYA		Soldan Sağa
16	Koşullu (ternary)	?:	Sağdan Sola
17	Atama	=, +=, -=, *=, /=, %=, <<=, >>=, &=, ^=, =	Sağdan Sola
18	Virgül	,	Soldan Sağa

Tablo 4.12: C++ İşleç Öncelik ve Birleşme Hiyerarşisi

4.9 Kayan Noktalı Sayı Hesapları

Kayan noktalı sayıların IEEE-754 standardında ikili tabanda tutulduğu 2.4 başlığında anlatılmıştı. Kodlarken onluk tabanda değerler verdiğimiz kayan noktalı sayılar, aslında arka planda ikilik tabanda tutulur. Bu durum yazdığımız kodlarda garip davranışların ortaya çıkmasına sebep olur. Buna örnek olarak; hemen hemen her programcının yaptığı ilk hata, aşağıdaki kodun amaçlandığı gibi çalışacağını varsaymaktır;

```
float total = 0.0f;
for(float a = 0.0f; a != 2.0f; a += 0.01f)
    total += a;
```

Acemi programcı bunun 0, 0.01, 0.02, 0.03, gibi artarak, 1.97, 1.98, 1.99 aralığındaki her bir sayıyı toplayacağını ve 199 sonucunu, yani matematiksel olarak doğru cevabı vereceğini varsayar. Bu kodda doğru yürümeyen iki şey olur:

1. Birincisi yazıldığı haliyle program asla sonuçlanmaz. `a` asla 2'ye eşit olmaz ve döngü asla sonlanmaz.
2. İkincisi ise döngü mantığını `a < 2.0f` şartını kontrol edecek şekilde yeniden yazarsak, döngü sonlanır, ancak toplam 199'dan farklı bir şey olur. IEEE-754 uyumlu işlemlerde, genellikle 2.0f yerine yaklaşık 201.f'e ulaşır. Bunun olmasının nedeni, kayan noktalı sayıların onluk tabanda kendilerine atanan değerlerin, **ikilik tabanda yaklaşık değerlerini temsil etmesidir**.

Bu duruma aşağıdaki klasik örnek verilir;

```
#include <iostream>
int main() {
    double a = 0.1;
    double b = 0.2;
    double c = 0.3;
    if(a + b == c)
        std::cout<<"Bu satır İcra Edilmez!"; //IEEE754 standardında işlem yaparsak.
    else
        std::cout<<"Kayan Noktalı Sayılar İkili Sayı Sisteminde Tutulduğundan Yaklaşık Olarak
        ↪ Hesaplanır." ;
}
```

Programcı olarak gördüğümüz şey onluk tabanda yazılmış üç sayı olsa da derleyicinin (ve altta yatan donanımın) gördüğü şey ikili sayılardır. 0.1, 0.2 ve 0.3, ona mükemmel bölmeyi gerektirdiğinden (ki bu onluk sayı sisteminde oldukça kolaydır), ancak ikili sayı sisteminde imkansızdır (bu sayılar, onluk sayı sistemindeki 1/3 sayısı gibi 0.333333333333333... şeklinde kesin olmayan biçimde depolanması gerektiğindendir);

```
int main()
{
```

